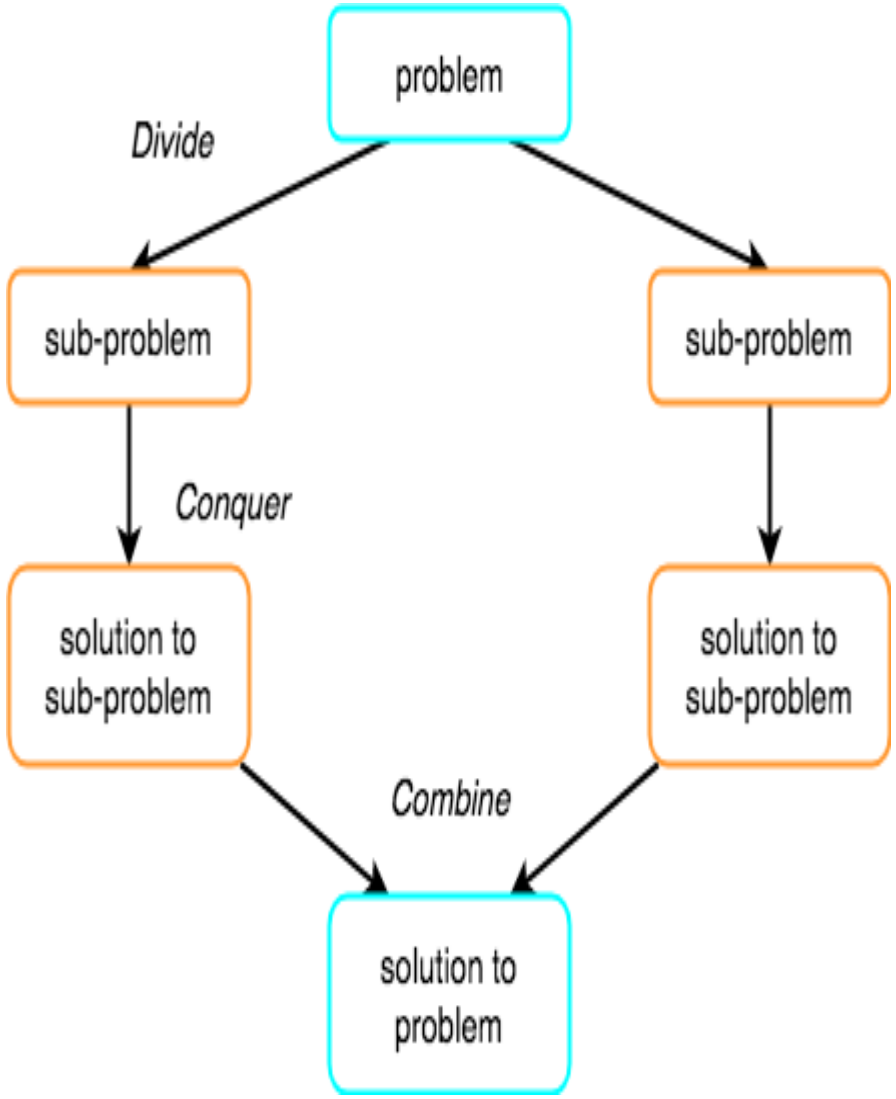
Merge Sort

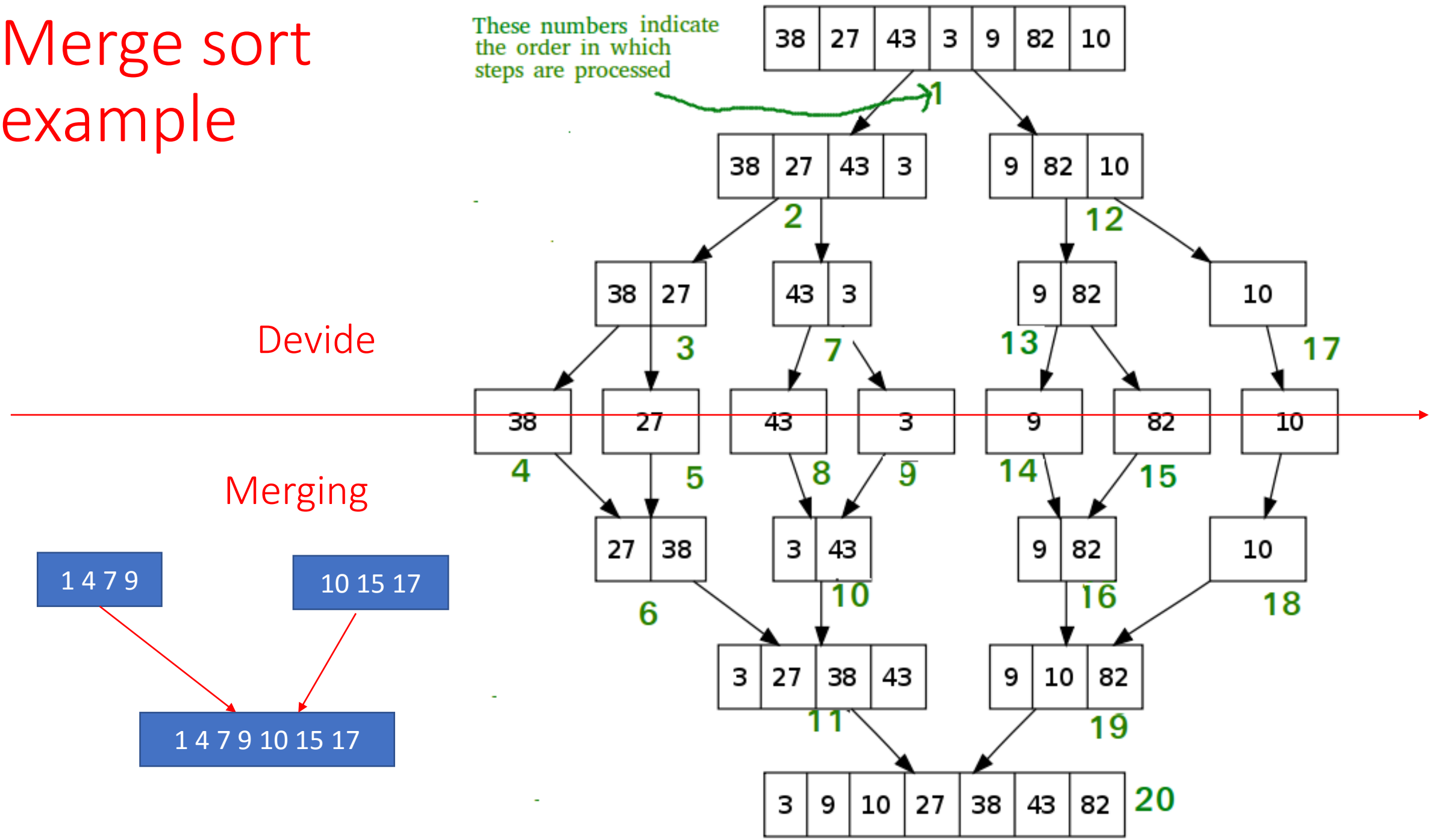
UNIT III

Divide and Rule

- The concept of Divide and Conquer involves three steps:
- **Divide** the problem into multiple small problems.
- **Conquer** the subproblems by solving them. The idea is to break down the problem into atomic subproblems, where they are actually solved.
- **Combine** the solutions of the subproblems to find the solution of the actual problem.



Merge sort example



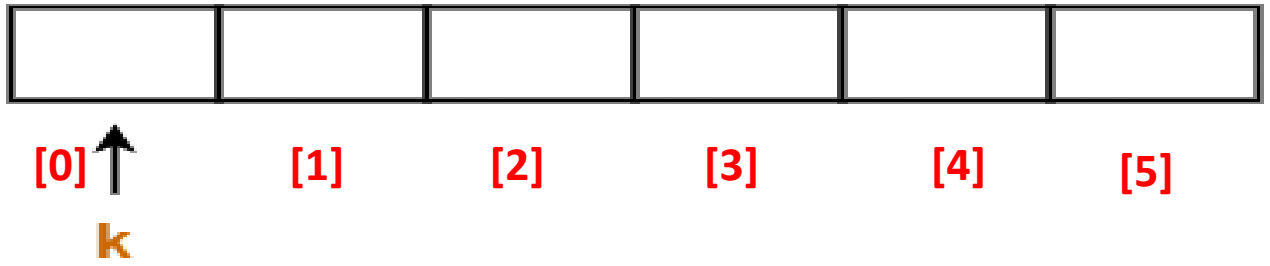
Algorithm Merging

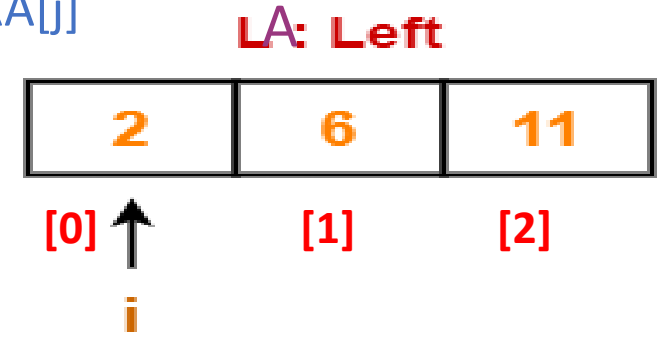
- Merge(LA,RA,MA)
- Len_LA= length(LA)
- Len_RA= length(RA)
- Set i=j=k=0
- While i < Len_LA and j < Len_RA
- Do
- If LA[i]≤ RA[j]
- then
- set MA[k]=LA[i]
- Inc k, i
- Else
- then
- set MA[k]=RA[j]
- Inc k, j
- End If.
- Done
- //Adding remaining elements from left sub array to array MA
- While i < Len_LA
- Do
- set MA[k]=LA[i]
- Inc k, i
- Done
- //Adding remaining elements from right sub array to array MA
- While j < Len_LR
- Do
- set MA[k]=RA[j]
- Inc k, j
- Done

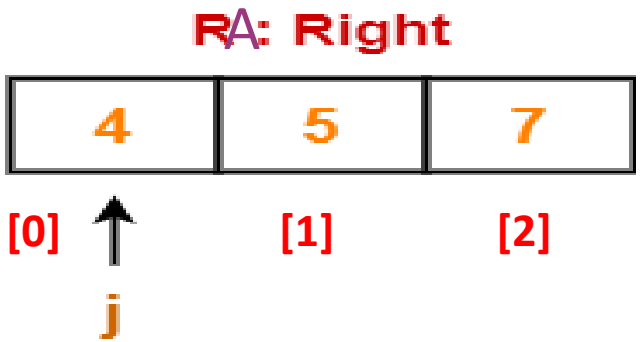
- Merge(LA,RA,MA)
- Len_LA= length(LA)
- Len_RA= length(RA)
- Set i=j=k=0
- While i < Len_LA and j < Len_RA
- Do
- `If LA[i]≤ RA[j]`
- `then`
- set MA[k]=LA[i]
- `Inc k, i`
- `Else`
- `then`
- set MA[k]=RA[j]
- `Inc k, j`
- End If.
- Done

i variable for left sub array
j variable for right sub array
k variable for sorted output array
i =0, j=0, k=0

Sorted output array: MA







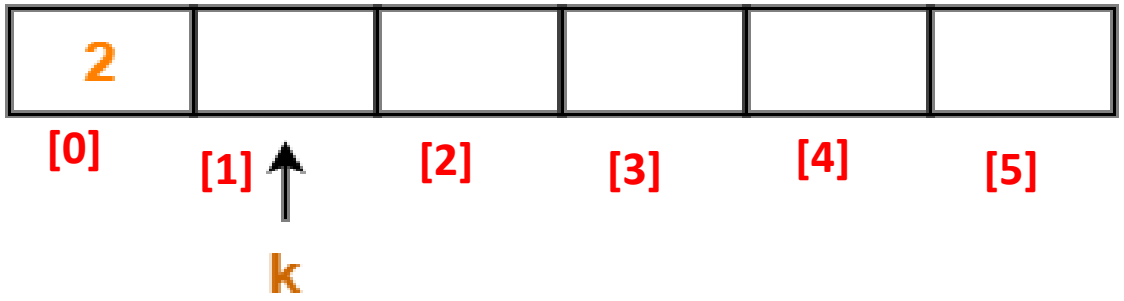
i =0, j=0, k=0

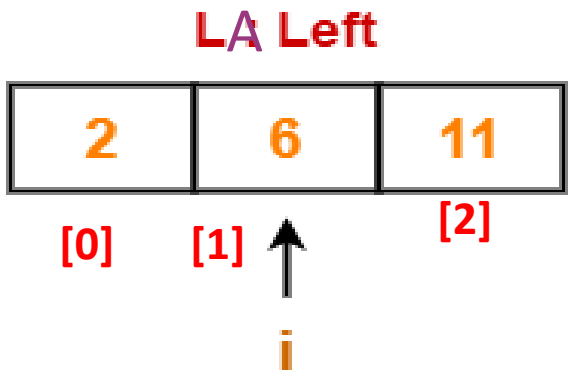
LA[0] < RA[0], MA[0]=LA[0],

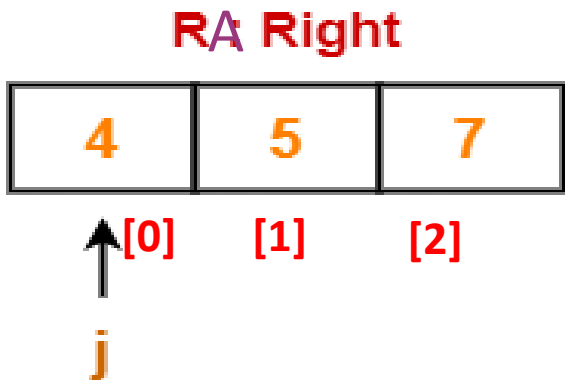
i++, k++

i=1,j=0,k=1

Sorted output array: MA







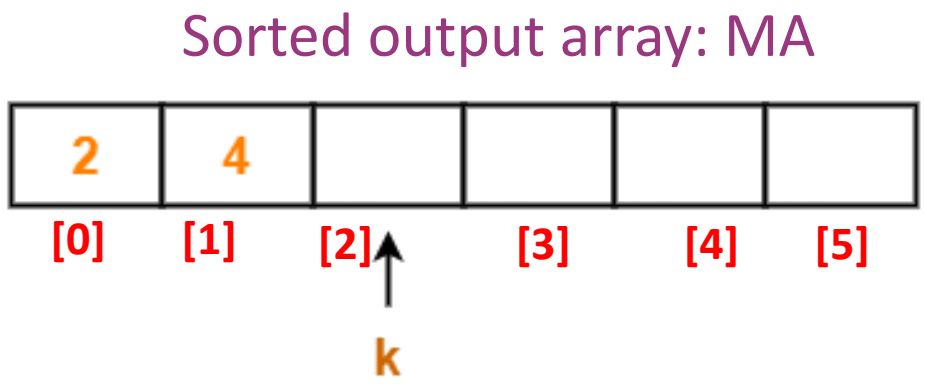
- Merge(LA,RA,MA)
- Len_LA= length(LA)
- Len_RA= length(RA)
- Set i=j=k=0
- While i < Len_LA and j < Len_RA
- Do
- `If LA[i]≤ RA[j]`
- `then`
- set MA[k]=LA[i]
- `Inc k, i`
- `Else`
- `then`
- set MA[k]=RA[j]
- `Inc k, j`
- End If.
- Done

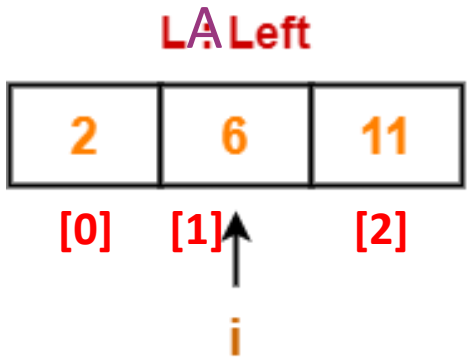
i =1, j=0, k=1

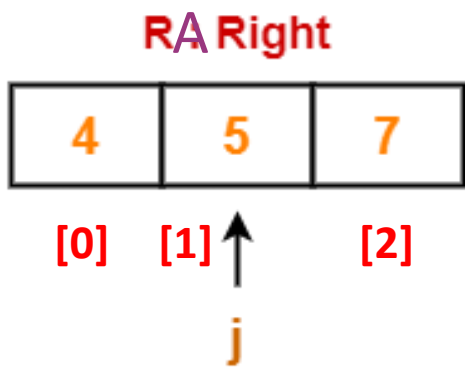
LA[1] > R[0], MA[1]=RA[0],

j++, k++

i=1,j=1,k=2







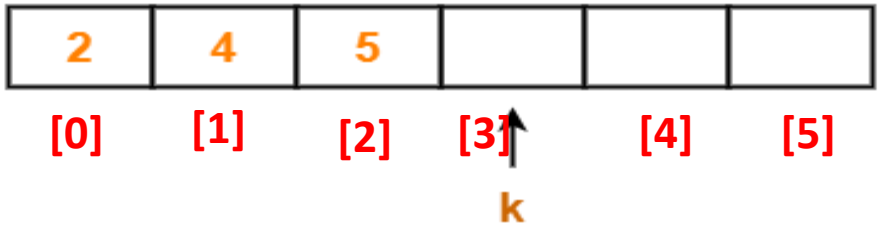
i =1, j=1, k=2

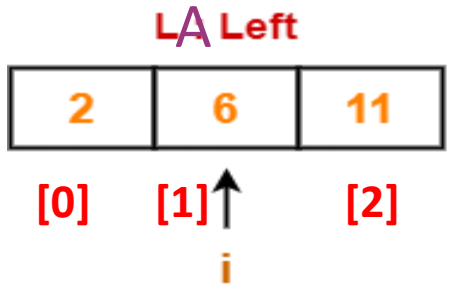
LA[1] > R[1], MA[2]=RA[1],

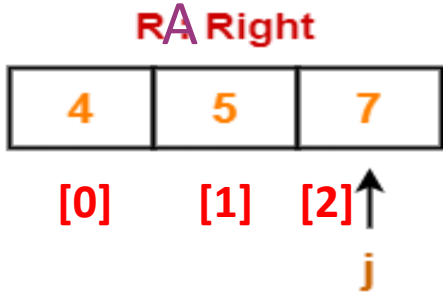
j++, k++

i=1,j=2,k=3

Sorted output array: MA





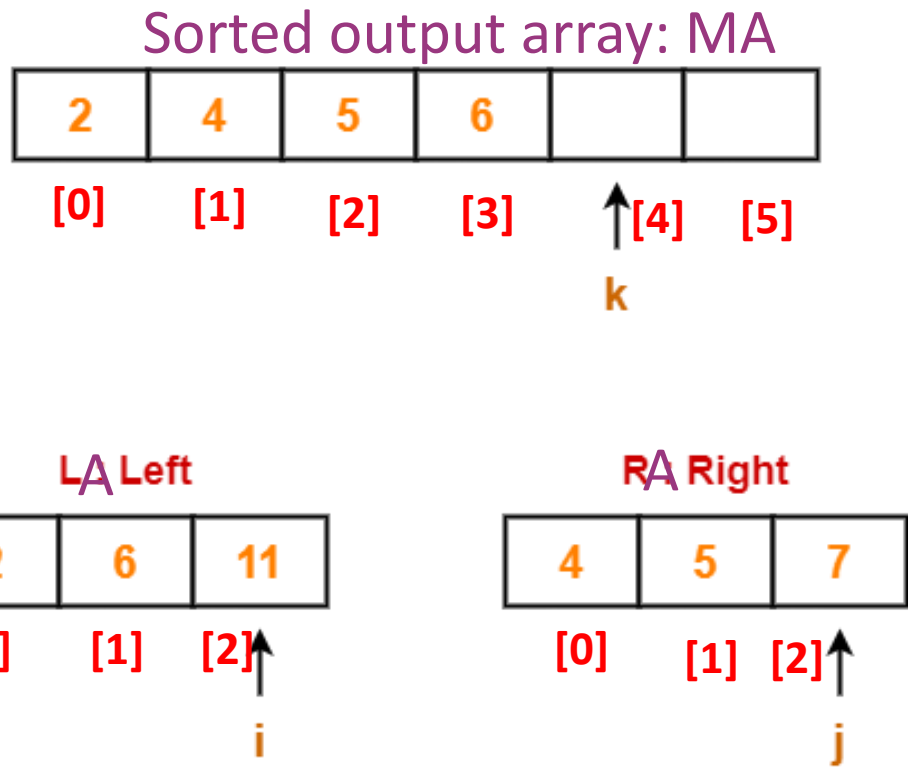


i =1, j=2, k=3

LA[1] < R[2], MA[3]=LA[1],

i++, k++

i=2,j=2,k=4



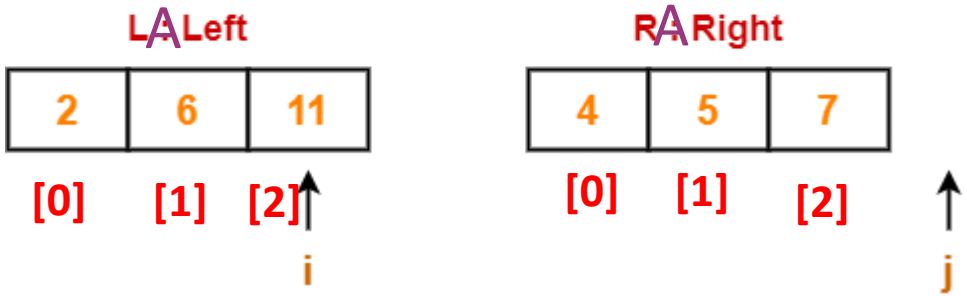
i =2, j=2, k=4

LA[1] > R[2], MA[4]=RA[2],

j++, k++

i=2,j=3,k=5

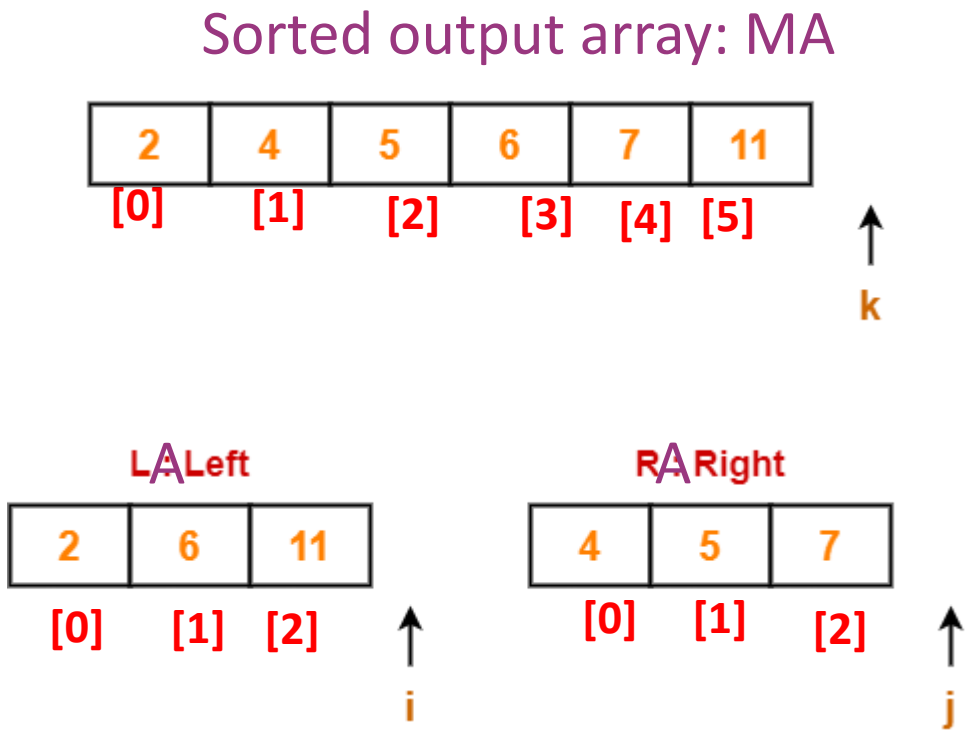
Sorted output array: MA



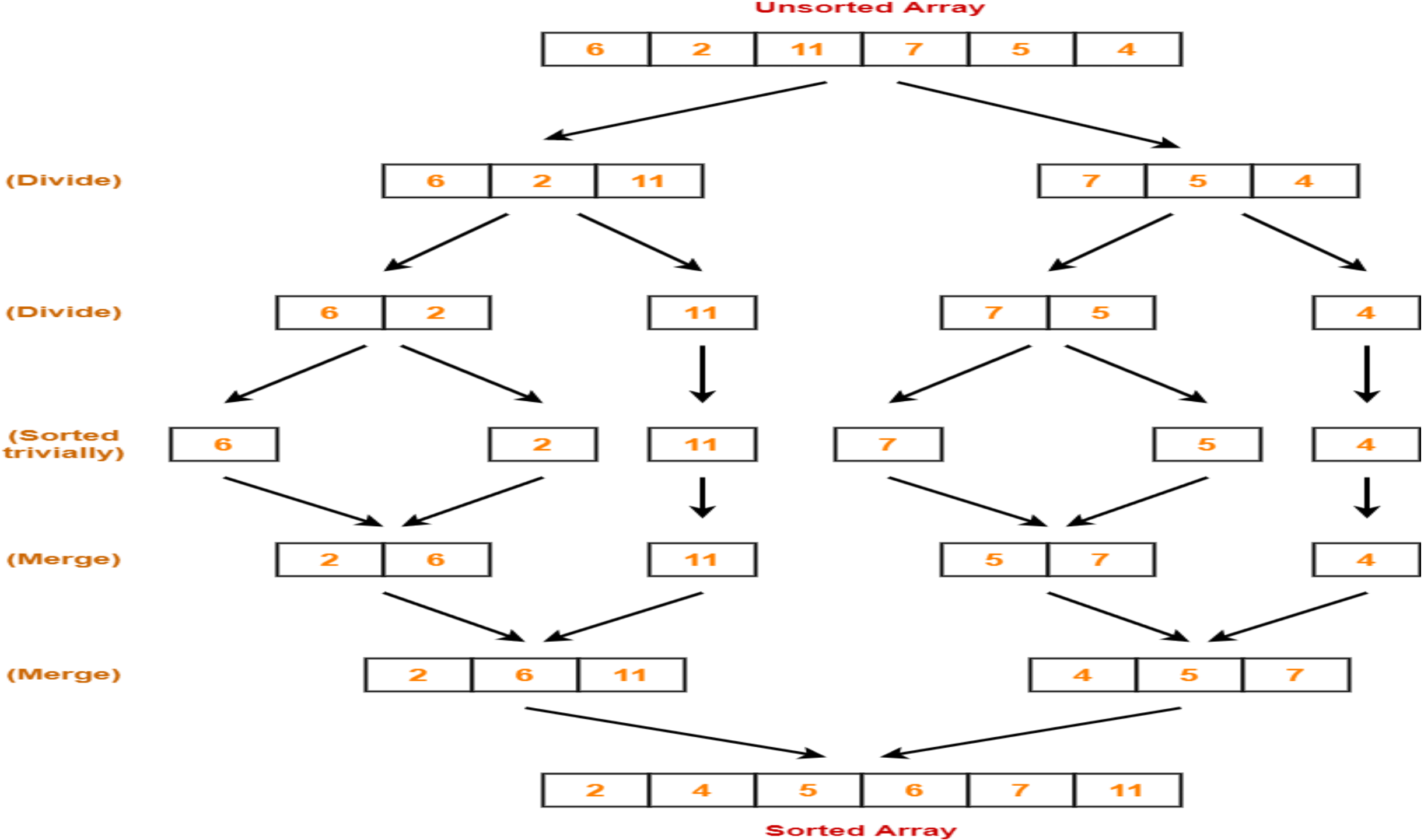
- Merge(LA,RA,MA)
- Len_LA= length(LA)
- Len_RA= length(RA)
- Set i=j=k=0
- While i < Len_LA and j < Len_RA
- Do
- If LA[i]≤ RA[j]
- then
- set MA[k]=LA[i]
- Inc k, i
- Else
- then
- set MA[k]=RA[j]
- Inc k, j
- End If.
- Done

i =2, j=3, k=5

Left sub array is added to MA



```
MergeSort(A)
{
    n = length(A)
    if n<2 return
    mid = n/2
    left = new_array_of_size(mid) // Creating temporary array for left
    right = new_array_of_size(n-mid) // and right sub arrays
    for(int i=0 ; i<=mid-1 ; ++i)
        left[i] = A[i] // Copying elements from A to left
    for(int i=mid ; i<=n-1 ; ++i)
        right[i-mid] = A[i] // Copying elements from A to right
    MergeSort(left) // Recursively solving for left sub array
    MergeSort(right) // Recursively solving for right sub array
    merge(left, right, A) // Merging two sorted left/right sub array to final array
}
```


Unsorted Array
6 2 11 7 5 4
(Divide)
6 2 11
7 5 4
(Divide)
6 2
11
7 5
4
(Sorted trivially)
6
2
11
7
5
4
(Merge)
2 6
11
5 7
4
(Merge)
2 6 11
4 5 7
2 4 5 6 7 11
Sorted Array

Time Complexity

- Selection sort consists of two nested loops
- It has O(n*log n) worst case time complexity

Analysis of merge sort algorithm

- Divide and conquer
- Recursive
- Stable
- Not In-place (It does not give O(1) order of time complexity).
- Space complexity $\Theta(n)$
- Time complexity $\Theta(n \log n)$

```
MergeSort(A)
{
    n = length(A)
    if n<2 return
c'' mid = n/2
    left = new_array_of_size(mid)
    right = new_array_of_size(n-mid)
    for(int i=0 ; i<=mid-1 ; ++i)
        left[i] = A[i]
    for(int i=mid ; i<=n-1 ; ++i)
        right[i-mid] = A[i]
    MergeSort(left)    T(n/2)
    MergeSort(right)   T(n/2)
    merge(left, right, A)  N*c')
}
```

$$T(n) = \begin{cases} c, & for\ n = 1 \\ 2T\left(\frac{n}{2}\right) + c'n + c'' & for\ n > 2 \end{cases}$$

$\left(\frac{n}{2^k}\right)$=1

$2^k = n$

$k = \log_2 n$

$T(n)$=2*T(n/2) +c'*n → $T(n)$=2 T(n/2)+c'n
= 2*{2*T(n/4)+c'n/2}+c'n

$$T(n) = 2\left\{2T\left(\frac{n}{4}\right) + c'\left(\frac{n}{2}\right)\right\} + c'n$$

$$T(n) = \left\{4T\left(\frac{n}{4}\right) + 2c' * \left(\frac{n}{2}\right)\right\} + c'n$$

$$T(n) = \left\{4T\left(\frac{n}{4}\right) + 2c'n\right\}$$

$$T(n) = 4\left\{2T\left(\frac{n}{8}\right) + c' * \left(\frac{n}{4}\right)\right\} + 2c'n$$

$$T(n) = 2^3 T(\frac{n}{2\text{^}3}) + 3c'n$$

$$T(n) = 16T(\frac{n}{16}) + 4c'n \rightarrow T(n) = 2^k T\left(\frac{n}{2^k}\right) + kc'n$$

$$T(n) = 2^k T\left(\frac{n}{2^k}\right) + kc'n$$

$$T(n) = 2^{\log_2 n} T\left(\frac{n}{2^{\log_2 n}}\right) + \log_2 n\, c'n$$

$$T(n) = 2^{\log_2 n} T(1) + \log_2 n\, c'n$$

$$T(n) = nC + C'n \log_2 n$$

$$T(n) = O(n \log n)$$

$T(n) = \Theta(n \log n)$ means: c1 n log n ≤T(n) ≤ c2 n log n, if n ≥n0

$T(n) = O(n \log n)$ means: T(n) ≤ c n log n, if n ≥n0

End